

Оценка требований к резервному копированию и резервированию

S3-Platform

20 июля 2026

Оценка требований к резервному копированию и резервированию

1. Основание и метод
2. Инвентаризация состояния
3. Классификация данных по восстановимости
4. Сохранность данных
5. Доступность и резервирование
6. Целевые показатели
7. Требования
8. Что необходимо исправить в диаграммах
9. Требуется уточнения

Оценка требований к резервному копированию и резервированию

1. Основание и метод

Оценка построена по текущему состоянию диаграмм репозитория, прежде всего `06-deploy.puml` (as-is топология) и `05-component.puml`, и уточнена фактическими данными об эксплуатации. Каждая находка сопровождается ссылкой на строку диаграммы либо пометкой «уточнено эксплуатацией», если источник — не модель.

Уточнения, полученные от владельца платформы:

- Redis, Postgres и Kafka развёрнуты на хостинге **Timeweb.cloud** с включённым сервисом ежедневного резервного копирования.
- Платформа **уже работает в продакшне**.
- `s3p-tape-service` использует Redis на Timeweb с включённым бэкапом.
- **Kafka выступает связью между `s3p-tape-service`, `s3p-n8n` и `s3p-node`**, то есть является транспортом между компонентами.
- Платформенный S3-бакет также защищён ежедневным сервисом резервного копирования Timeweb.
- Каждый `s3p-node` имеет **собственное локальное хранилище**; хранилища между узлами не связаны, конкурентной перезаписи нет.

Эти уточнения существенно меняют картину по сравнению с тем, что изображено на диаграммах, и это само по себе является главной находкой для репозитория документации — см. находку 1.

Итог смещения акцентов. Вопросы **сохранности** данных в значительной степени закрыты провайдерскими бэкапами. Практически весь остаточный риск сосредоточен в **доступности**: резервирования нет ни у одного критичного компонента, и в продакшне это уже не отложенная задача.

2. Инвентаризация состояния

Хранилище	Где	Что хранит	Резервное копирование	Резервирование
s3p-database (Postgres 16, sppIntegratedDB)	Timeweb.cloud	схемы nodes, tasks, plugins, sources, ml, documents, analytics, control, score, users	ежедневно, Timeweb (не отражено на диаграмме)	нет (1 экземпляр)
Redis	Timeweb.cloud	сессии, ленты tape:<user>	ежедневно, Timeweb (не отражено)	нет
Kafka	Timeweb.cloud	сообщения в пути между tape-service, n8n и s3p-node; офсеты потребителей	ежедневно, Timeweb — решает не ту задачу, см. находку 7	нет данных о replication factor

Хранилище	Где	Что хранит	Резервное копирование	Резервирование
n8n metadata (Postgres 12, том db_data)	Compose-хост	воркфлоу n8n, учётные данные, история запусков	pg_dump nightly → том pg_backups на том же хосте (06-deploy.puml:41,91)	нет
Локальное хранилище s3p-node	по одному на узел	кеш скачанных артефактов плагинов	не требуется (кеш)	по узлу, изолировано
Платформенный S3-бакет	Timeweb.cloud	plugins/<repo>/ {plugin.xml, src/*}	ежедневно, Timeweb; версионирование объектов не отражено	пересобираемо из GitHub
Секреты s3-platform-credentials- {database,s3,github}, ghcr-secret	объекты кластера	доступы ко всем внешним системам	нет	нет
GHCR	внешний реестр	образы контейнеров	не требуется	пересобираемо

3. Классификация данных по восстановимости

Требования к бэкапу задаются не важностью сервиса, а тем, можно ли данные воссоздать.

Класс	Данные	Воссоздаваемо?	Последствия потери
А. Невосстановимо	score.score — вердикты и комментарии экспертов	Нет. Это человеческий труд	Безвозвратная потеря основной ценности платформы
А	users	Нет	Потеря учётных записей и связей с оценками
А	Воркфлоу n8n	Нет. Создаются вручную в UI (05-component.puml:82)	Сбор материалов останавливается, восстановление — недели
А/В	documents — собранные материалы	Частично. Повторный обход возможен, но источники изменчивы, платны и удаляются	Утрата исторического среза, обнуление аналитики
В. Конфигурация	Секреты	Нет, если не хранятся вне кластера	Блокирует восстановление целиком
В	tasks, nodes, control, analytics, ml, sources	Преимущественно операционные / производные	Деградация, восстановимо
С. Производное / транзитное	Сообщения и офсеты Kafka	Транзитные. Исходное состояние остаётся в Postgres	Потеря событий в пути, рассинхронизация до сверки
С	Redis tape:<user>	Да. Воркер регенерирует из БД (04b:39-41)	Кратковременная деградация
С	Redis-сессии	Нет, но ценность низкая	Разлогинивание экспертов
С	Локальное хранилище узла	Да. Скачивается из S3 при каждой задаче (03a:33)	Отсутствуют
Д. Пересобираемое	Артефакты плагинов в S3, образы GHCR	Да, через CI из GitHub; S3 дополнительно копируется Timeweb	Простой выполнения задач до восстановления или пересборки

4. Сохранность данных

Находка 1 — модель существенно расходится с продакшном

Это главная находка для репозитория, назначение которого — описывать платформу. Диаграммы расходятся с фактической эксплуатацией минимум в пяти местах:

Что показывает модель	Что в действительности
У <code>s3p-database</code> нет резервного копирования; единственное ребро бэкапа ведёт к <code>n8n_pg</code> (<code>06-deploy.puml:91</code>)	Postgres на Timeweb с ежедневным бэкапом
Redis без признаков персистентности и бэкапа (<code>06-deploy.puml:52</code>)	Redis на Timeweb с ежедневным бэкапом
Kafka отсутствует во всех девяти диаграммах	Kafka связывает <code>tape-service</code> , <code>n8n</code> и <code>s3p-node</code>
Один PVC внутри Deployment (<code>replicas: 3</code>) (<code>06-deploy.puml:24-28</code>)	У каждого узла собственное изолированное локальное хранилище
Кластер <code>demo-s3p-dev</code> , теги <code>:dev</code> , образ <code>3.0.3-beta</code>	Продакшн-контур

Практический риск здесь не теоретический. Любой, кто планирует восстановление или оценку рисков по этим диаграммам, спланирует не то: решит, что основная база не копируется, не увидит транспортный слой между тремя ключевыми сервисами и заложит несуществующую проблему конкурентной записи в общий том. Именно это и произошло в первой редакции настоящей оценки.

Наиболее весомая часть расхождения — отсутствие Kafka. Диаграммы описывают интеграцию трёх сервисов через другие механизмы: `s3p-node` опрашивает `dbTask.relevant(node)` каждые 5 секунд (`04a:21-23`), воркер `tape-service` слушает `LISTEN/NOTIFY` (`04b:37`), `n8n` ходит в `s3p-simple-api` по HTTP (`04a:15`). Если те же связи сегодня несёт Kafka, то модель описывает не дополнительный слой, а вытесненный. Как именно Kafka соотносится с этими путями — заменяет их или дополняет — определяет часть выводов ниже и вынесено в раздел 9.

Отдельно отмечу именование. `demo-s3p-dev` и теги `:dev` на продакшн-контуре — самостоятельный операционный риск: они провоцируют недооценку критичности среды при любых работах.

Находка 2 — RPO 24 часа на невозстановимых данных в продакшне

Ежедневное резервное копирование означает RPO около 24 часов. Для класса А это значит, что до суток экспертной разметки — то есть человеческого труда, который невозможно воссоздать, — может быть потеряно безвозвратно.

Это ключевой остаточный риск сохранности. Для `tasks` или `nodes` суточный RPO приемлем: они операционные. Для `score.score` и `users` он приемлем ровно настолько, насколько платформа готова попросить экспертов переразметить день работы.

Решение — PITR или архивирование WAL для `s3p-database`, что снижает RPO до минут. Уточнить, доступен ли PITR в тарифе Timeweb, либо организовать непрерывное архивирование самостоятельно.

Находка 3 — бэкап n8n находится в том же домене отказа

Уточнение про Timeweb касается Redis, Postgres и Kafka. База метаданных n8n — это `postgres:12` в Compose на отдельном хосте (`06-deploy.puml:36–46`), и её бэкап пишется в том `pg_backups` **на том же хосте**.

Потеря хоста уничтожает базу и её резервные копии одновременно. При этом там лежат воркфлоу n8n — данные класса A, созданные вручную и не воспроизводимые из git (`05-component.puml:82`).

Это самый слабый по сохранности узел платформы после уточнений. Требуется копия за пределами хоста.

Находка 4 — провайдерский бэкап не равен проверенному восстановлению

Включённый сервис бэкапа фиксирует, что копии создаются. Он не отвечает на вопросы, которые определяют реальный RTO: сколько занимает восстановление, до какой точки, кто имеет права на операцию и проверялась ли она хоть раз.

Непроверенный бэкап — это гипотеза, а не гарантия. Для продакшн-системы с невосстановимыми данными учебное восстановление обязательно, и его результат должен быть зафиксирован.

5. Доступность и резервирование

После уточнений именно здесь сосредоточен основной остаточный риск. Бэкапы защищают от потери данных, но ничего не делают со временем простоя, а платформа работает в продакшне.

Находка 5 — резервирование обратно пропорционально критичности

Единственный компонент с избыточностью — `s3p-node Deployment (replicas: 3)` (`06-deploy.puml:24`). При этом именно его отказ поглощается наиболее мягко: задачи durable лежат в Postgres со статусом `scheduled`, а `PushTrigger` опрашивает `dbTask.relevant(node)` каждые 5 секунд (`04a:21–23`), поэтому любая выжившая реплика подберёт работу.

Все компоненты, отказ которых означает полный отказ функции, существуют в единственном экземпляре:

Компонент	Последствие отказа в продакшне
Postgres (s3p-database)	Полная остановка платформы
Kafka	Разрыв связи между tape-service, n8n и s3p-node одновременно
VM s3p-tape-service	Полная остановка экспертной оценки
Compose host (n8n + simple-api + n8n_pg)	Полная остановка сбора материалов
Static hosting	Низкий риск, SPA не имеет состояния

Усилия по резервированию вложены в слой, который лучше всех переживает отказ, и не вложены ни в один слой, где отказ является полным.

Появление Kafka в этом перечне заметно повышает ставку. Транспорт, связывающий три ключевых сервиса, концентрирует риск: его отказ бьёт не по одной функции, а по всем связям сразу.

Находка 6 — VM tape-service совмещает весь путь аннотации

На одной VM размещены `tape-service-api`, `tape-service-worker` и Redis (`06-deploy.puml:49-57`). Отказ VM одновременно убирает REST API, WS-пуш и регенерацию ленты — ни один элемент пути экспертной оценки не выживает.

Уточнение про Timeweb смягчает часть картины: если Redis вынесен на Timeweb, а не живёт на самой VM, то сессии переживут пересоздание машины. Это расхождение с `06-deploy.puml:52`, где Redis изображён внутри узла VM, и его нужно отразить в диаграмме.

Находка 7 — транспортному слою нужна репликация, а не суточная копия

Это относится и к Kafka, и к Redis, и является, пожалуй, самым практически значимым выводом об их защите.

Содержимое обоих сервисов транзитно или производно. Ленты Redis воссоздаются воркером из Postgres (`04b:39-41`). Сообщения Kafka — это события в пути, тогда как исходное состояние остаётся в Postgres.

Отсюда следует, что ежедневный бэкап здесь решает не ту задачу, а при восстановлении может навредить. Поднять Kafka из вчерашней копии — значит вернуть устаревшие офсеты и переиграть уже обработанные сообщения, то есть заново запустить задачи, которые давно выполнены. Восстановление Redis из суточного дампа вернёт устаревший набор кандидатов там, где регенерация из БД дала бы корректный.

Что действительно требуется этому слою:

- Kafka — `replication factor` не ниже 3 и `min.insync.replicas` не ниже 2, чтобы отказ брокера не приводил ни к простоям, ни к потере сообщений.
- Kafka — `durability`-хранение офсетов потребителей и осознанно выбранный `retention`, достаточный, чтобы потребитель пережил окно недоступности и дочитал накопленное.
- Redis — персистентность и/или реплика ради доступности.

Ни один из этих параметров не отражён ни в диаграммах, ни в уточнениях; их следует зафиксировать явно.

Находка 8 — событийный путь через LISTEN/NOTIFY недолговечен

Регенерация ленты запускается уведомлением `pg_notify('score_added')` (`04b:37`, `06-deploy.puml:95`). Postgres не накапливает уведомления для отключённых слушателей.

Если воркер недоступен в момент срабатывания NOTIFY — во время деплоя, перезапуска или сетевого разрыва — событие теряется безвозвратно, и лента пользователя не регенерируется до следующей оценки.

Здесь важно уточнение про Kafka, и оно работает в пользу платформы. Kafka, в отличие от NOTIFY, сохраняет сообщения для отключённых потребителей, поэтому перевод этого пути на Kafka закрывает описанный пробел по сути, а не обходит его. Если такой перевод уже выполнен, находка неактуальна и её следует снять; если LISTEN/NOTIFY сохраняется параллельно — она в силе. Это вынесено в раздел 9 как первый по значимости вопрос.

Независимо от ответа остаётся ограничение для планов резервирования: промоушен реплики Postgres разрывает все регистрации LISTEN. Любая схема НА обязана включать переподключение воркера и последующий сверочный проход. На диаграмме есть WORKER_INTERVAL_SECONDS=60 (APScheduler, 06-deploy.puml:51), что может служить таким проходом, но диаграмма не сообщает, выполняет ли периодический запуск сверку или только плановую работу.

Находка 9 — у секретов нет пути восстановления

Четыре секрета существуют только как объекты кластера (06-deploy.puml:29–32). cloud-core управляет платформой декларативно через ArgoCD (06-deploy.puml:78), но секреты в git обычно не хранят, а внешнего хранилища секретов диаграмма не показывает.

При потере кластера провайдерский бэкап Postgres не поможет: платформу нечем будет запустить. Это тот класс дефекта, который превращает часовое восстановление в двухдневное. Отмечу, что появление Kafka добавляет к этому набору ещё один комплект доступов, который тоже должен иметь источник вне кластера.

Находка 10 — откат ошибочного релиза плагина неизбирателен

Это находка наименьшей severity в перечне, и уточнения дополнительно её смягчают.

GitHub Actions пишет артефакты напрямую в платформенный S3-бакет (06-deploy.puml:100), а s3p-node читает оттуда при каждом запуске задачи (04a:32–37). Ошибочный релиз перезаписывает работающий плагин.

Путей восстановления теперь два, и оба существуют: пересборка из GitHub и суточная копия Timeweb. Остаётся вопрос не наличия отката, а его избирательности. Суточная копия — это снимок бакета целиком, поэтому откат к ней затрагивает и все остальные плагины, выпущенные в тот же день. Версионирование объектов даёт точечный откат одного артефакта без побочного эффекта для соседних релизов.

Практический вывод: специально заводить защиту здесь не требуется, но версионирование объектов стоит включить как дешёвую страховку от неизбирательного восстановления.

6. Целевые показатели

Класс данных	RPO сейчас	RPO целевой	RTO целевой
A — оценки, пользователи	~24 ч (Timeweb daily)	≤ 15 мин (PITR / WAL)	≤ 4 ч
A — воркфлоу n8n	~24 ч, копия на том же хосте	≤ 24 ч, но обязательно вне хоста	≤ 4 ч
A/B — documents	~24 ч	≤ 1 ч	≤ 8 ч

Класс данных	RPO сейчас	RPO целевой	RTO целевой
B — секреты	не покрыто	0 (версионироваться вне кластера)	≤ 30 мин
C — Kafka	~24 ч (нерелевантно)	не применимо; вместо RPO — RF ≥ 3 и durable-офсеты	≤ 5 мин доступности
C — Redis	~24 ч (нерелевантно)	не применимо, данные производные	≤ 5 мин доступности
D — артефакты S3, GHCR	~24 ч (Timeweb daily), плюс пересборка из GitHub	0 при версионировании объектов	≤ 1 ч

RTO по всем строкам указан как целевой, поскольку фактический RTO неизвестен до проведения учебного восстановления (находка 4).

7. Требования

Уровни отражают характер риска, а не срок: платформа в продакшне, поэтому оба первых уровня актуальны сейчас.

Уровень 1 — сохранность

1. Провести и задокументировать учебное восстановление каждого хранилища на Timeweb, зафиксировав фактические RTO и точку восстановления (находка 4).
2. Обеспечить копию бэкапа n8n за пределами Compose-хоста (находка 3).
3. Снизить RPO для `score.score` и `users` через PITR или архивирование WAL (находка 2).
4. Вынести секреты во внешний источник, включая доступы к Kafka (находка 9).

Уровень 2 — доступность

1. Зафиксировать и при необходимости поднять параметры отказо-устойчивости Kafka: `replication factor ≥ 3`, `min.insync.replicas ≥ 2`, durable-офсеты, осознанный `retention` (находка 7).
2. Добавить горячий standby для `s3p-database` (находка 5).
3. Разнести `tape-service-api`, `tape-service-worker` и Redis, убрав совмещённый домен отказа (находка 6).
4. Включить персистентность и/или реплику Redis ради доступности (находка 7).
5. Обеспечить сверочный проход и корректное переподключение воркера после failover (находка 8).
6. Зарезервировать хост n8n либо перевести воркфлоу в воспроизводимый из git вид, что попутно выводит их из класса A (находки 3, 5).
7. Включить версионирование объектов S3-бакета ради избирательного отката отдельного артефакта (находка 10).

Уровень 3 — при росте нагрузки

1. Автоматический failover Postgres и разнесение по зонам.

8. Что необходимо исправить в диаграммах

Расхождения модели с продакшном из находки 1 устраняются правкой диаграмм:

- добавить Kafka как транспорт между `s3p-tape-service`, `s3p-n8n` и `s3p-node` в `05-component.puml` и `06-deploy.puml`, а в `01-archimate3-layered.puml` — как элемент технологического слоя;
- показать при этом, какие из существующих связей Kafka заменяет: опрос `dbTask.relevant`, `LISTEN/NOTIFY` или HTTP-путь через `s3p-simple-api`;
- перенести Redis из узла VM в контур Timeweb (`06-deploy.puml:52`);
- показать Postgres, Redis и Kafka как управляемые сервисы Timeweb с признаком ежедневного бэкапа;
- заменить единственный PVC на изолированное локальное хранилище каждого узла (`06-deploy.puml:24-28`);
- отразить отсутствие standby явно, чтобы оно читалось как решение, а не как умолчание;
- привести именование в соответствие со статусом среды либо снабдить диаграмму пометкой, что `demo-s3p-dev` — продакшн;
- добавить заметку с RPO/RTO по каждому хранилищу.

Естественная форма — отдельная диаграмма `07-deploy-to-be` того же разрешённого типа `deploy`, со строкой в таблице `README.md` согласно `CLAUDE.md`. Добавление Kafka затрагивает и последовательности `04a / 04b`, если транспорт заменил изображённые там пути.

Отмечу, что `README.md:29` называет базу «managed Postgres», и уточнения подтверждают: прав `README`, а `06-deploy.puml` устарел. Расхождение следует устранять правкой диаграммы, а не `README`.

9. Требуется уточнения

1. Заменяет ли Kafka изображённые пути интеграции — опрос `dbTask.relevant`, `LISTEN/NOTIFY`, HTTP через `s3p-simple-api` — или дополняет их. От этого зависит актуальность находки 8 и объём правок в `04a / 04b`.
2. Текущие `replication factor`, `min.insync.replicas` и `retention` Kafka (находка 7).
3. Доступен ли PITR в используемом тарифе Timeweb (находка 2).
4. Фактическое время восстановления и глубина хранения копий Timeweb (находка 4).
5. Выполняет ли `WORKER_INTERVAL_SECONDS=60` сверку состояния или только плановую работу (находка 8).
6. Способ подачи секретов в кластер и наличие источника вне кластера (находка 9).
7. Включено ли версионирование объектов S3-бакета в дополнение к суточной копии (находка 10).
8. Размещён ли Redis на Timeweb или на самой VM tape-service — диаграмма и уточнение расходятся (находка 6).